

PX4128 Data Analysis Block C

Background Reading

Written by Dr Paul Clark

adapted by Dr Steve Maddox

November 12, 2019

Contents

1	Time series data and spectral analysis	2
1.1	Time series data	2
1.2	Spectral Analysis	3
1.3	Fourier frequencies and Aliasing	5
1.4	Fourier Transforms	6
1.5	The power spectrum, or periodogram	9
1.6	Noise in the power spectrum	10
1.7	Spectral Leakage	10
1.8	Data Smoothing	14
1.9	Background Subtraction	15
1.10	Fourier Filtering	16
1.11	Auto-correlation	17
1.12	Correlated noise	18
1.13	Cross-correlation	20
1.14	Convolution	21

Chapter 1

Time series data and spectral analysis

1.1 Time series data

This chapter will deal with cases where the measured quantity is function of time, i.e. $x(t)$, and the techniques that are commonly employed to deal with such cases. Typical examples of time series data include light curves in astronomical data, or the type of general signal processing that can arise in sound engineering and system monitoring. Note however than many of the concepts introduced in this chapter are applicable to data sets where the dependent variable is something other than time!

In general a time series is simply a set of measurements $\{x_j\}$ taken at a series of times $\{t_j\}$. This is assumed to represent an underlying continuous function of time.

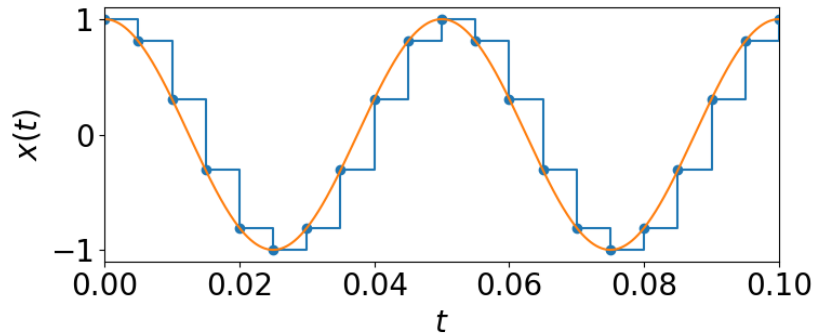


Figure 1.1: A sine wave sampled with uniform time intervals.

For equally spaced time series, $\Delta t = t_{j+1} - t_j = \text{constant}$, so each bin covers from $t_j = j\Delta t$, to $(j+1)\Delta t$. If we have N bins, then J ranges from 0 to $N-1$, and the total time covered by the series is $T = N\Delta t$.

Having equally spaced time series where $\Delta T = \text{constant}$ allows us to use standard tools of spectral analysis. A word of caution though: the methods outlined below are generally

applicable only to data sets where the measurements are evenly spaced in time, and there are no gaps. The spectral analysis of non-evenly spaced data is beyond this course, although in many cases it is possible to create evenly spaced data by interpolation.

One further aspect that is generally assumed for simple time series is that they represent ‘stationary processes’. A **stationary** process is one that satisfies the condition that the descriptions of the data are insensitive to a shift s in time t . For example, the mean

$$\bar{x} = \sum (x(t_1), x(t_2), \dots, x(t_N)) / N$$

and the shifted mean

$$\bar{x}_s = \sum (x(s + t_1), x(s + t_2), \dots, x(s + t_N)) / N$$

are the same, for any arbitrary shift s . This is essentially equivalent to saying that each series is a fair sample of the underlying population

1.2 Spectral Analysis

A very common reason to measure a time series is because we believe that there may be some periodic phenomenon present in the system under study. For example we might be analysing the intensity of a star as function of time, in the hope to discover a planetary transit. In this case, we are typically trying to determine i) What is the period T , or frequency $f = 1/T$, of the phenomena, and ii) what is the amplitude of the associated signal. The steps that we go through to determine these parameters are called **spectral analysis**.

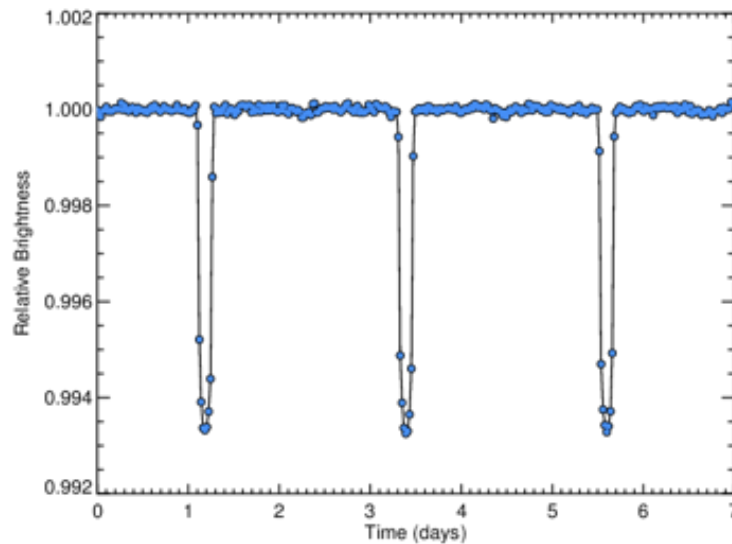


Figure 1.2: An example light curve for star with a transitting planetary system.

The basic idea behind spectral analysis is the same as that behind a Fourier transform: any function, or shape, can be approximated with an infinite series of sinusoidal waves of differing

period and amplitude. If a function truly is periodic, then we can get away with perhaps only a few of these sinusoidal waves to build a complete description. The Fourier Transform of a signal gives the amplitudes and phases of the sinusoids that make up the signal. Because we have discrete time points, we are typically dealing with discrete Fourier transforms or **DFTs**, and in particular FFTs.

Recall that the general form of a wave is given by,

$$x(t) = A \cos(2\pi f t + \phi) \quad (1.1)$$

As usual, t moves from 0 through to T the cosine functions moves from 0 to 2π , and thus completes the cycle f times. The phase, ϕ controls the shift w.r.t. to the standard 0 to 2π cycle. The term A controls the amplitude of the wave.

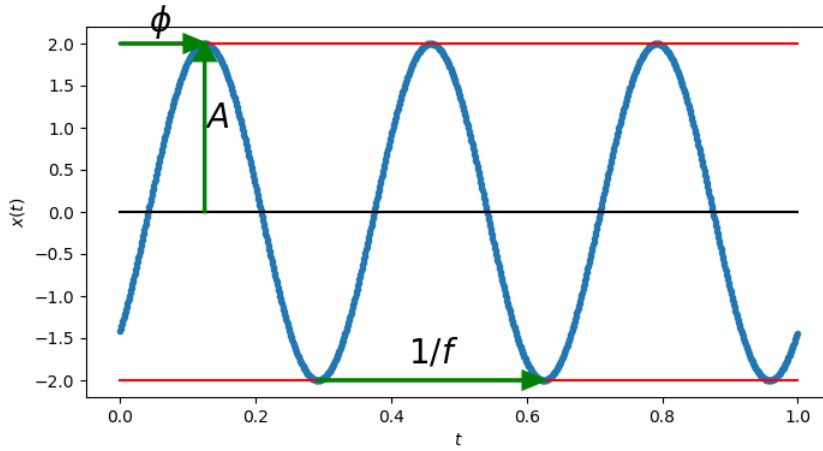


Figure 1.3: A cosine wave with amplitude A , phase ϕ and frequency f .

We can now make use of a trigonometric identity to simplify the above expression,

$$A \cos(2\pi f t + \phi) = \alpha \cos(2\pi f t) + \beta \sin(2\pi f t) \quad (1.2)$$

where

$$\alpha = A \cos(\phi) \quad (1.3)$$

and

$$\beta = -A \sin(\phi) \quad (1.4)$$

So a simple cosine wave can be generalised by a mixture of sin and cos terms, with the phase becoming part of the amplitude

It is common to write this using complex notation,

$$A \cos(2\pi f t + \phi) = \frac{(\alpha - i\beta)}{2} e^{i2\pi f t} + \frac{\alpha + i\beta}{2} e^{-i2\pi f t} \quad (1.5)$$

The real part of the amplitudes give the cos term, and the imaginary parts the sin term. The phase of the complex amplitude is ϕ

1.3 Fourier frequencies and Aliasing

In what follows, we will assume that our time series comprises a even number N of time bins of width Δt , such that the total epoch in the series is $N\Delta t = T$. In spectral analysis, we decompose the function into a linear sum of such wave functions, each with a different frequency f . These frequencies are chosen (i.e. probed/tested) and the goal of the analysis is see how strong each of the waves is: i.e. we are looking to determine the **amplitude**. For a Fourier transform, we use sine and cosine as the wave functions, and chose harmonic frequencies such that an integer number of cycles of each wave will fit within the length of the data. If we have N data points in our time series, where N is a even number, then we can approximate the data with the following sum,

$$x(t_j) = \sum_{k=1}^{N/2} [\alpha_k \cos(2\pi f_k t_j) + \beta_k \sin(2\pi f_k t_j)] \quad (1.6)$$

where the frequencies $f_k = \frac{k}{T}$, are the harmonic frequencies that we are using to probe the time series. For $j = 1$, we have $f_1 = \frac{1}{T}$ which is just the fundamental frequency of the time series – if our measured time series were to be a complete cycle of some sort, this is frequency it would have. The extreme case when $k = N/2$ gives $f_{N/2} = \frac{N/2}{T} = \frac{N/2}{N\Delta t} = \frac{1}{2\Delta T}$, which is known as **the Nyquist frequency**, and denotes the highest frequency that can be ‘resolved’ (or at least properly identified) in the time series.

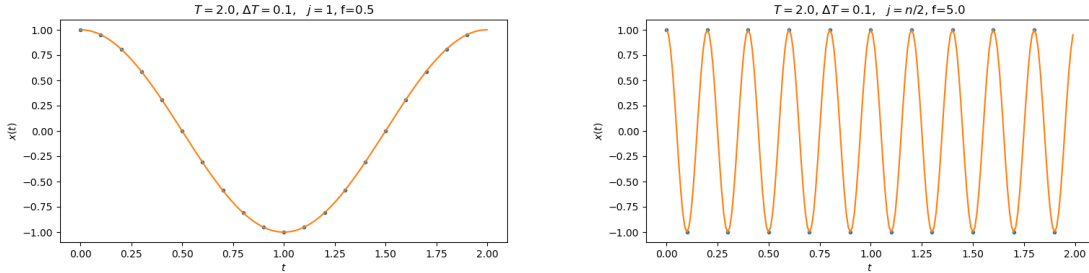


Figure 1.4: The fundamental frequency and the Nyquist frequency

Frequencies higher than $1/(2\Delta T)$ appear as lower frequencies, because of the spacing of data samples is more that half a cycle. In general a wave of frequency f_k sampled at a rate ΔT appears the same as a wave of frequency $f_k - \frac{1}{\Delta t} = f_{k-N}$. Specific interesting specific cases are:

- $f_{N/2} = \frac{N}{2T}$ is equivalent to $f_{-N/2} = -\frac{N}{2T} = -f_{N/2}$. Note that $\cos(2\pi f_{N/2} t_j) = \pm 1$ and $\sin(2\pi f_{N/2} t_j) = 0$
- $f_{N-1} = \frac{N-1}{T}$ is equivalent to $f_{-1} = -\frac{1}{T} = -f_1$. So that $\cos(2\pi f_{N-1} t_j) = \cos(2\pi f_{-1} t_j) = \cos(2\pi f_1 t_j)$ but $\sin(2\pi f_{N-1} t_j) = \sin(2\pi f_{-1} t_j) = -\sin(2\pi f_1 t_j)$
- $f_N = \frac{N}{T}$ is equivalent to $f_0 = 0$. So that $\cos(2\pi f_N t_j) = \cos(2\pi f_0 t_j) = 1$ and $\sin(2\pi f_N t_j) = \sin(2\pi f_0 t_j) = 0$

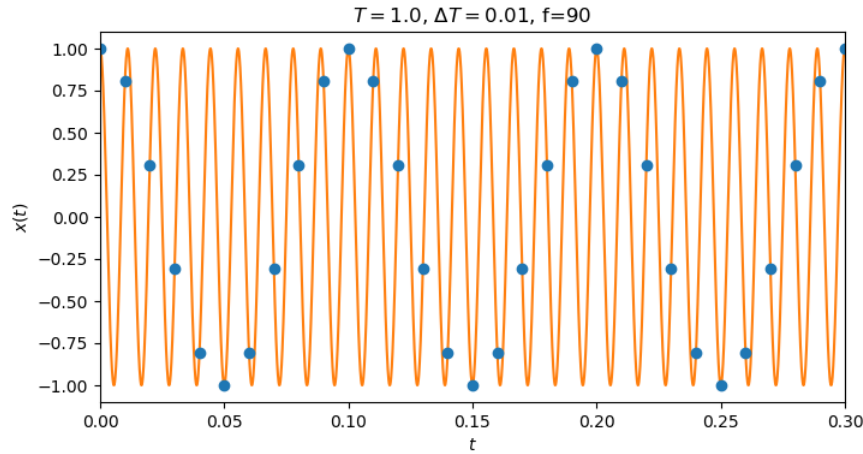


Figure 1.5: Aliasing. With a sampling rate of 100Hz, a cosine wave with frequency 90Hz is indistinguishable from a cosine wave with frequency 10Hz

1.4 Fourier Transforms

Since the frequencies f_k are known (i.e. they are decided upon by the data analyst), we have N unknowns (we have $N/2$ terms of α_j and β_j apiece), and we have N data points, and so an exact solution can be found. You will also note that this is an example of a **linear model**, and so the techniques for standard linear regression can be applied to this problem.

Perhaps a more complete description of the time series than is permitted by 1.6 is to allow j to run from 0 instead of 1. In this case, we see that this allows us to model an offset in the amplitude. Taking this into account, and noting that the $n/2$ th member of the series has no sine term (it goes to zero), we can then re-write 1.6 as,

$$x(t_j) = \alpha_0 + \sum_{k=1}^{N/2-1} [\alpha_k \cos(2\pi f_k t_j) + \beta_k \sin(2\pi f_k t)] + \alpha_{N/2} \cos(\pi f_{N/2} t_j) \quad (1.7)$$

Using complex notation, the two sums can be combined into one

$$x(t_j) = \sum_{j=-N/2}^{N/2-1} C_k e^{2\pi i f_k t_j} \quad (1.8)$$

where

$$C_k = \begin{cases} \frac{1}{2} (\alpha_k - i\beta_k) & \text{if } k \geq 0 \\ \frac{1}{2} (\alpha_k + i\beta_k) & \text{if } k < 0 \end{cases} \quad (1.9)$$

Using standard linear regression on the expressions for α and β , and assuming that the errors were normal and homoscedastic, we can write the fit parameters as

$$\alpha_0 = \hat{x}(t) = \frac{1}{N} \sum_{j=1}^N x(t_j) \quad (1.10)$$

$$\alpha_k = \frac{1}{N} \left[\sum_{j=1}^N x(t_j) \cos(2\pi f_k t_j) \right] \quad (1.11)$$

$$\beta_k = \frac{1}{N} \left[\sum_{j=1}^N x(t_j) \sin(2\pi f_k t_j) \right] \quad (1.12)$$

$$\alpha_{N/2} = \sum_{j=1}^N (-1)^j x(t_j) / N \quad (1.13)$$

In the complex notation, this becomes

$$C_k = \frac{1}{N} \sum_{j=0}^{N-1} x(t_j) e^{-2\pi i f_k t_j} \quad (1.14)$$

One useful feature of these fit parameters is that they are **orthogonal**: i.e. the value of 1 parameter, say α_k does not affect the value of the others, e.g. β_k or α_{k+1} , etc. In fact orthogonality is a mathematical requirement of the Fourier series. This means that we are free to remove the k the component of series before fitting for the $k + 1$ th component, etc. In practise one usually fits for the lowest frequency harmonic first, subtracts this from the data series, and moves on to find the next component. This process is known as **spectral pre-whitening** of the data.

Although this application of linear regression is possible, and permits us to treat the case where hetroscedastic errors are present on the measurements of $x(t_j)$, we often employ a different technique to find the coefficients: namely a **Fast Fourier Transform** or **FFT**. This can make use of built in libraries in c/Fortran/python, etc to very quickly compute the discrete Fourier transform of $\{x(t_j)\}$, and return the amplitude of each of the frequencies in the series. These libraries typically use the complex notation to return the Fourier coefficients.

Given an array of N data points $\{x_j\}$, an FFT function will return N amplitudes $\{A_k\}$, one for each of the frequency components in the series. Using the complex notation

$$A_k = \sum_{j=0}^{N-1} x(t_j) e^{-2\pi i f_k t_j} = N C_k = \frac{1}{2} N (\alpha_k - i\beta_k). \quad (1.15)$$

So $C_k = A_k/N$. Note, some packages may use different conventions for the normalisation, using N , $\sqrt{N}$ or combinations including 2π . This is the convention used by `numpy`.

If the data are real valued, then the Fourier coefficients are symmetrical, so that

$$\text{real}(A_k) = \text{real}(A_{-k}) \quad (1.16)$$

and

$$\text{imag}(A_k) = -\text{imag}(A_{-k}) \quad (1.17)$$

FFT routines usually return an array of complex amplitudes for N frequencies corresponding to $f_0 = 0$ to $f_{N-1} = (N-1)/T$. It is often easier to interpret this as a range of frequencies from $-N/2$ to $+N/2$, with 0 in the middle. To do this, the second half of the array of amplitudes must be shifted to the beginning of the array, and the first half shifted to the end. There is a python routine `fftshift` which re-organizes the array to do this.

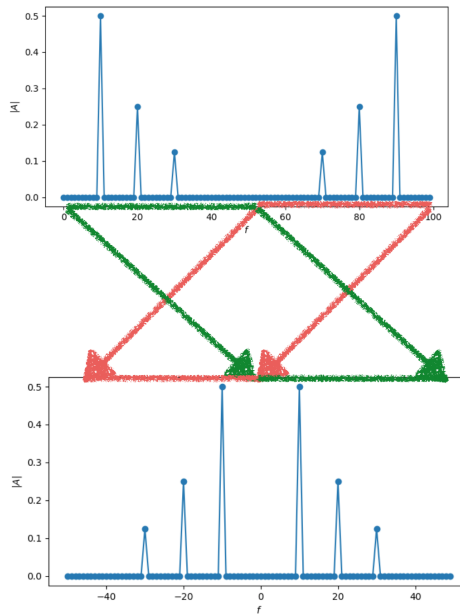


Figure 1.6: The `numpy` routine `fftshift` moves the second half of an array to the start, and the first half to the end. This re-arranges the array of fourier amplitudes so that the first element corresponds to $f_{-N/2}$, the middle element to $f_0 = 0$, and the last element $f_{+N/2}$.

For real valued data, the symmetry of the Fourier components means that the total power at any frequency is split equally between the positive and negative frequencies.

The FFT routine calculates the set of Fourier coefficients that represent the data in Fourier space

$$\{A_k\} = \text{FFT}(\{x_j\}) \quad (1.18)$$

We can also define the inverse FFT, which can be written as FFT^{-1} . This calculates the data that would have produced a given set of Fourier coefficients

$$\{x_j\} = \text{FFT}^{-1}(\{A_k\}) \quad (1.19)$$

1.5 The power spectrum, or periodogram

Once the coefficients α_k and β_k have been obtained as the real and imaginary parts of C_k , it is useful to construct the **power spectrum**, or periodogram of the data. The power in a signal is defined as

$$P = \lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T |x|^2 dt \quad (1.20)$$

If the signal consists of a single cosine wave with amplitude α the power is $\alpha^2/2$. The power as a function of frequency is given by the square of the amplitude of the Fourier coefficients,

$$S(f_k) = |C_k|^2 = \frac{1}{4}(\alpha_k^2 + \beta_k^2) \quad (1.21)$$

The symmetry between f_{-k} and f_k means that $S(f_k) = S(f_{-k})$. For the case of a single sine wave of amplitude α , the sum over frequencies f_{-k} to f_k gives a total power of $\alpha^2/2$, as expected from the time-domain definition of power.

A plot of the power as a function of frequency can help identify the dominant frequencies in the time series. Examples are shown in Figure 1.7. By examining the amplitudes of the various frequency components in the data, we can identify the key frequencies that are present in the data. If the periodogram is relatively featureless, then it means that there are no dominant modes in the signal – quite possibly we just have white noise.

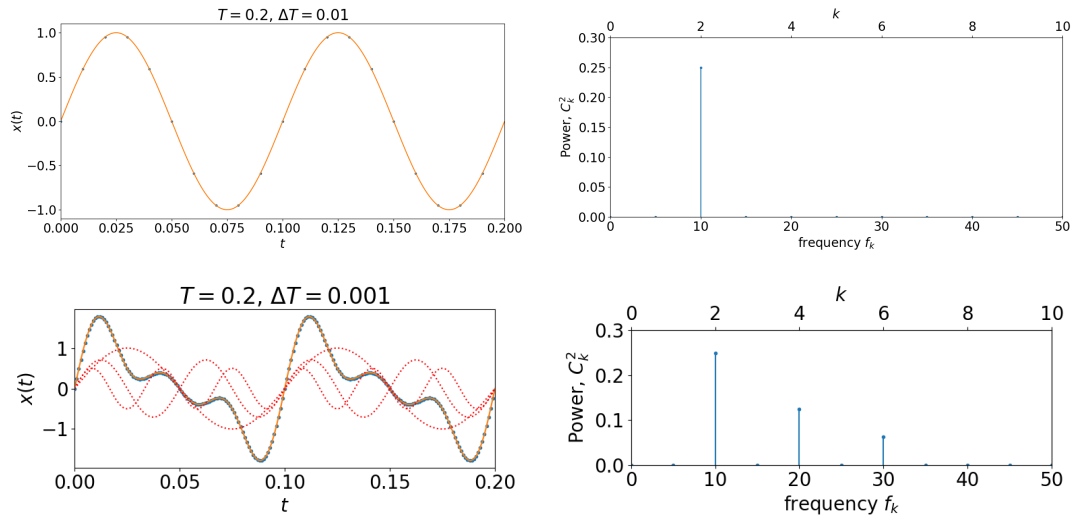


Figure 1.7: Top left: a series of 20 measurements with $\Delta t = 0.01$ and $T = 0.2$ with a signal $x = \sin(2\pi 10t)$. Top right: the corresponding power spectrum, showing one component at 10Hz. Bottom left, the signal with two extra frequency components, so $x = \sin(2\pi 10t) + \frac{1}{\sqrt{2}} \sin(2\pi 20t) + \frac{1}{2} \sin(2\pi 30t)$. Bottom right: the corresponding power spectrum with three components. Note that the power in each component is $\propto$ amplitude² of the contributing sine wave.

1.6 Noise in the power spectrum

If the data have random noise that is uncorrelated between each point power is added to the data at all frequencies. If we look at the single sine wave signal as before, but add noise

$$x(t_j) = \sin(2\pi 10 t_j) + \epsilon_j \quad (1.22)$$

where each ϵ_j is a random number sampled from a noise distribution usually assumed to be a Gaussian.

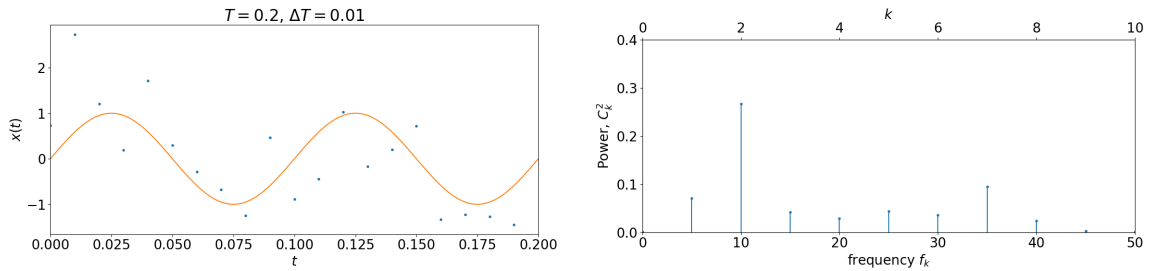


Figure 1.8: A single sine wave, $x(t_j) = \sin(2\pi 10 t_j)$ with Gaussian noise added. The noise adds to the power spectrum at all frequencies

The noise adds power to the power spectrum at all frequencies. The actual values of the frequency components will depend on the actual random values of the noise, but the expected power spectrum of uncorrelated random noise is a constant, independent of frequency. It is often called white noise.

1.7 Spectral Leakage

Although the procedure described above is extremely powerful, it is worth noting that it can be sensitive to spectral leakage, which spreads power from the true frequency to other frequencies.

The fundamental reason is that our time series is finite in length, and the total duration determines the spacing of the Fourier frequencies. The decomposition into components depends on choosing frequencies that fit an integer number of cycles within the total duration of the data, $f_k = k/T$. Therefore the spacing of the frequencies from f_k to f_{k+1} is $1/T$. As $T \rightarrow \infty$, the frequency spacing tends to zero, and we would have a perfectly sampled Fourier transform. In practice T must be finite, so we always have discrete frequency samples.

Also the Fourier analysis is mathematically equivalent to analysing a data set where the actual data from $t = 0$ to $t = T$ is reproduced periodically to infinity. This means that any difference between the start of the data and the end of the data will introduce sharp a discontinuity.

If the duration of the time series is not an integer multiple of the period of the main component in the signal, as shown in Figure 1.9, then there are two potential effects. First, the frequency of the true signal is not exactly one of those included in $\{f_k\}$, so the power at the true frequency will be spread between the nearest allowed frequencies. Second, the periodically replicated data has a discontinuity which further spreads the power to many frequencies.

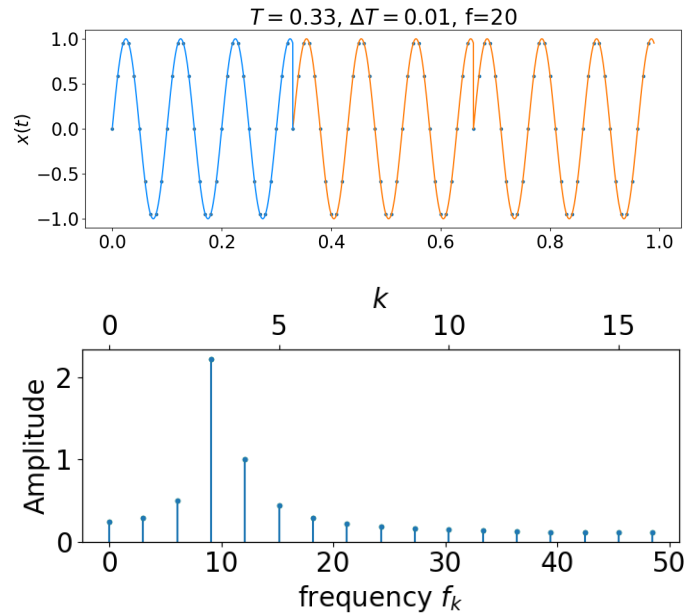


Figure 1.9: Top: A signal from a single sine wave of frequency 10Hz in a time series covering 1/3 of a second, as shown in blue. The frequency of the signal is such that the duration of the time series does not cover an integer number of periods, then there is a discontinuity between the start and end of the data. Bottom: The power spectrum of the time series has power at all frequencies, even though the input signal consists of a single sine wave.

This means that power is spread over all frequencies in the power spectrum, even though the input signal consists of a single sine wave which has power at only one frequency. Power has leaked from the true frequency to neighbouring frequencies. This problem can be particularly important if multiple components that are close together in frequency become smeared out in the power spectrum.

One way to get around this problem is to force your time series to look more periodic. A simple way to do this is to multiply the data in the series by a **window function** that is zero at the start of the series, rises and then falls back to zero at the end. In this way, the Fourier transform ‘sees’ a function that does not have any sharp discontinuities. This process, known as **tapering** is used extensively in spectral analysis to improve the power spectrum. The case where we use all the data, i.e. do not apply a window to the data, is actually known as a **square window**. Generally speaking, the choice of window function represents a trade-off between sharpness of the central peak, and the amount of power in the leakage and how far it extends away from the central peak.

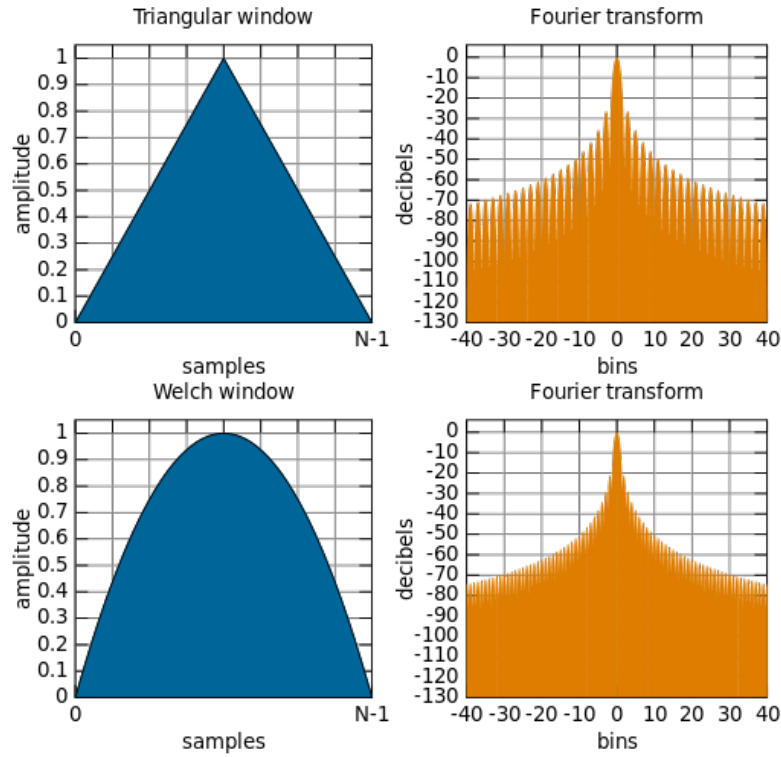


Figure 1.10:

One of the most common window functions is the **Bartlett** window (they are usually named after someone!). This is a triangular window and has the form,

$$w_j = 1 - \left| \frac{j - N/2}{N/2} \right| \quad (1.23)$$

where once again j represents the time bin (with width Δt). The Bartlett window is shown in Figure 1.10, and we see that this is essentially just a triangular function that peaks at $j = n/2$ and falls to zero at both $j = 0$ and N . The Bartlett window is good at reducing the power in the side-lobes and is recommended for cases where the data train is probably not just a simple periodic function.

Another popular choice is the **Welch window**, which is similar in form to the Bartlett:

$$w_j = 1 - \left(\frac{j - N/2}{N/2} \right)^2 \quad (1.24)$$

but is a little flatter in the middle. This is also shown in Figure 1.10. Again this is generally a good choice of window function, as it effectively reduces the side-lobes.

We show the effect of applying a Bartlett window in Figure ??.

Another common situation where spectral leakage happens is when there is a long timescale change in the calibration of e.g a linear drift such as

$$x(t) = \sin(2\pi f t) + A t \quad (1.25)$$

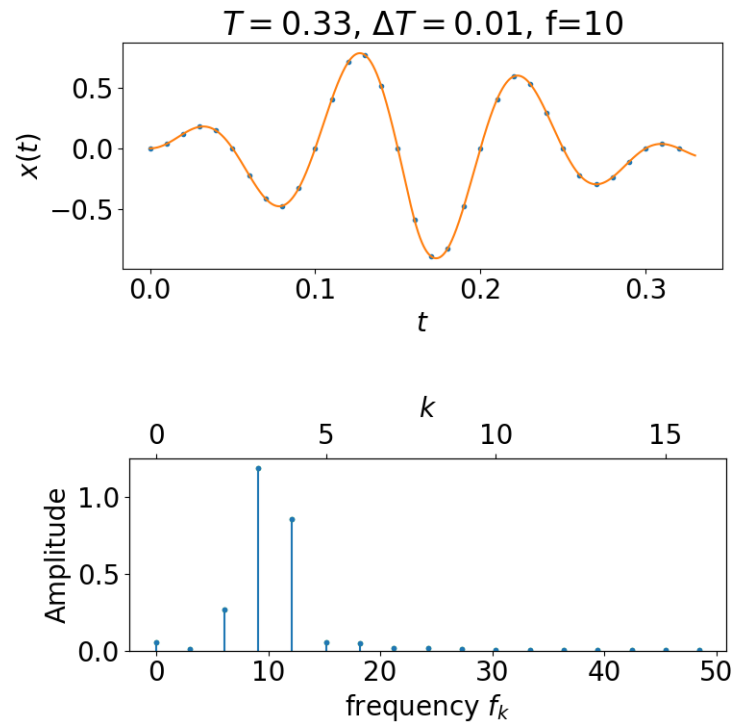


Figure 1.11: The same as Figure 1.9 but with a Bartlett window.

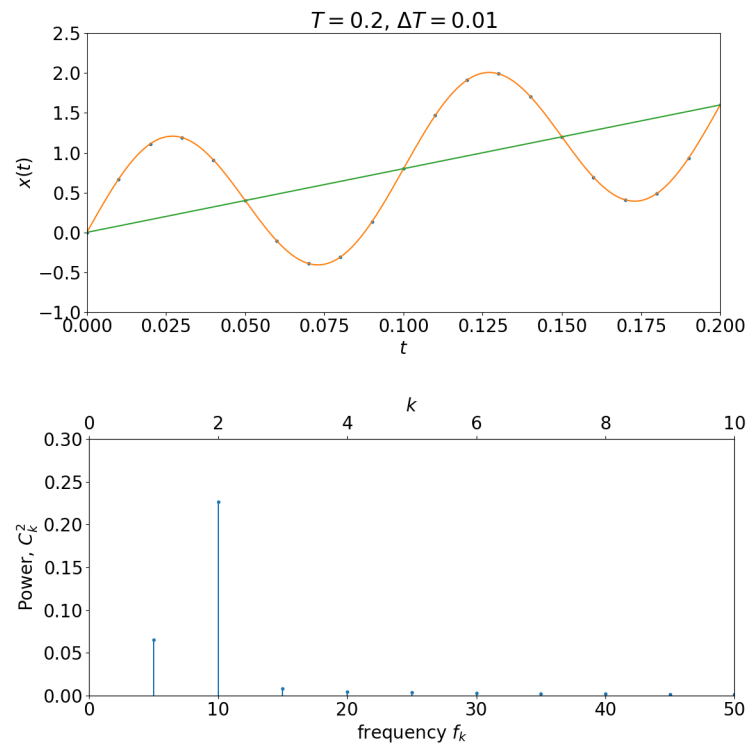


Figure 1.12: Top: a single sine wave with a background drift. Bottom: The corresponding power spectrum of the time series. Even though the input signal consists of a single sine wave there is power at a range of frequencies, and particularly at low frequencies.

When we perform our spectral analysis of the data, the decomposition will attempt to represent the linear rise with some combination of low frequency components. If our underlying sinusoidal frequency is also low, then its contribution to the periodogram will be corrupted, simply because it is not the only feature in the data that can be explained by low frequency components. We can avoid this problem simply by removing the ‘trend’ in data before we perform our spectral analysis. In essence, we can remove the background. This is typically done by the smoothing or spline fitting that we describe later, although in certain cases we can also do very low frequency filtering to achieve the same result.

1.8 Data Smoothing

We will now briefly discuss the concept of **data smoothing**. However, first a warning: smoothing data will remove information from your data set. If the smoothing is done to reduce what you know to be noise, then the process can be relatively safe. For example, if we know that a particular instrument generates **white** (i.e. normally distributed) noise in measuring a signal $f(t)$, one can in principle smooth the time series to remove this instrument noise. Note that this type of normally distributed noise can be separated from the denominator in the χ^2 statistic. But you need to be careful, as often the noise that you see in a signal is due to real variability in the system being measured.

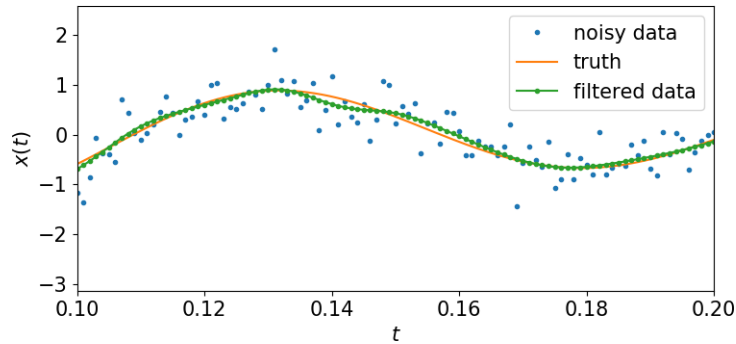


Figure 1.13: An example of smoothing data using a moving mean.

The most simple type of data smoothing is just the **central moving average**. In this case, one defines a window in the dependent variable, which we will call h , and simply average the points that lie within $\pm 1/2h$ of each data point t_j , i.e.

$$\hat{x}(t_j) = \frac{1}{N_{\pm 1/2h}} \sum_{t_j \geq t_j - 1/2h}^{t_j \leq t_j + 1/2h} x(t_j) \quad (1.26)$$

where $N_{\pm 1/2h}$ denotes the number of data points that lie within the window of width h . Although very simple, this method of smoothing has the advantage that it is straightforward to combine the heteroscedastic errors on each point, using the standard method of weighted

averages. A very similar alternative is to use a moving median instead of the moving mean. The median is more robust to extreme outliers in the data, and so is more effective at rejecting outliers. The penalty is that the expected standard deviation of the median is 1.25 higher than that for the mean.

1.9 Background Subtraction

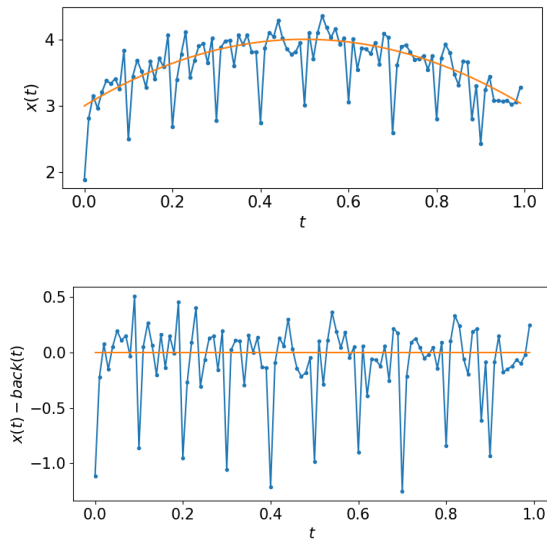


Figure 1.14: An example of removing a background from a spectrum

A common application of smoothing is to remove large-scale trends in the data set. Take for example, the case of a blackbody radiation field that is being passed through a gas of, say sodium. In such a case, we will get a series of absorption lines at the point corresponding to the emission lines of the sodium atom. The width of these lines will depend on the temperature of the sodium gas (via thermal broadening), as so one may wish to measure the width of the absorption features to get an estimate of the gas temperature. To do so, one ideally wants to remove the underlying blackbody spectrum, so that the absorption line can be fit from a level continuum, rather than trying to fit to the varying shape of the underlying background. In principle, smoothing allows you to create a new data set that follows the shape of the main underlying features, such as the blackbody curve. We can then **subtract** this shape from our data set to get a new data set that contains only the features that we are interested in. See Figure 1.14 for an example.

Another common approach to follow large-scale trends in the data is to fit a high-order **polynomial** to the data. Although this is very straightforward, the polynomials tend to be poorly constrained near the ends of the data, and often this is where correcting the trends is most important.

Yet another method is to fit a **cubic spline** to the data. A cubic spline is basically a series of 3rd order polynomials – i.e. cubic functions – that are ‘spliced’ together at the ends of each section. In doing this, the fitting function matches both the function and its first and second derivatives at the joining point, so that each cubic function merges smoothly into the next. Although this fitting procedure can be performed using linear regression, in practice it is easier to use an off-the-shelf function for polynomial fitting. The procedure normally involves first identifying a series of **knots** – positions at which the spline is to be fit, for example the case shown in Figure 1.15. Typically these are chosen to be regularly spaced within the depend variable’s range, and are often evaluated by averaging the data around the point in the manner discussed above. Once the spline’s coefficients have been computed, we can then use a spline interpolation function (which normally comes as part of the spline fitting package) to compute the values of $f(x)$ at each of the original data points x_j . The spline’s estimate for $f(x_j)$ can then be subtracted from the observed value, to remove the large-scale features. One advantage of the cubic spline over the high order polynomial fit is that the cubic functions will behave much better at the the ends of the data set, exhibiting less of the ‘flailing’ that we mentioned earlier.

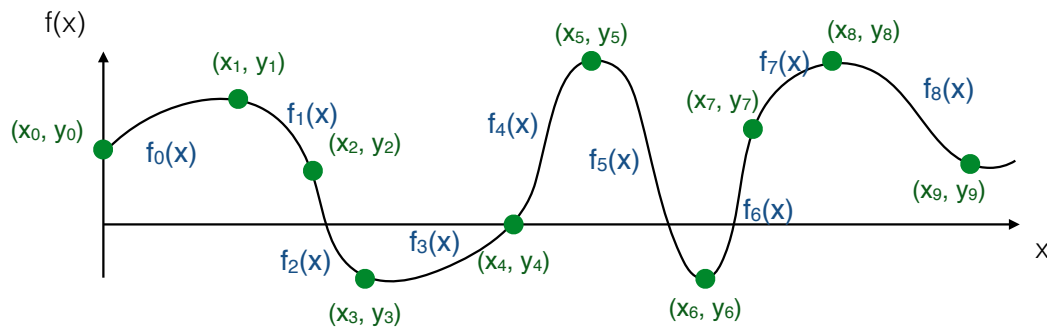


Figure 1.15: Fitting a cubic spline – a piecewise 3rd-order polynomial – to a set of knots

One feature of all of the above methods, is that they permit us to create a data set with constant steps Δt , even when the original data set did not. This can have several advantages when it comes to performing spectral analysis on the data set (see below), although one also needs to be mindful of the errors (i.e. bias) that this may introduce.

1.10 Fourier Filtering

Both smoothing and background removal can be achieved by multiplying the Fourier transform by a window function, and then taking the inverse Fourier transform. A low-pass filter suppresses high frequency components and removes small-scale variations, effectively smoothing the data. It does not affect large-scale trends in the data.

A high-pass filter suppresses low frequency components and removes large-scale trends, effectively removing any long time-scale background. On it’s own it does not smooth the data,

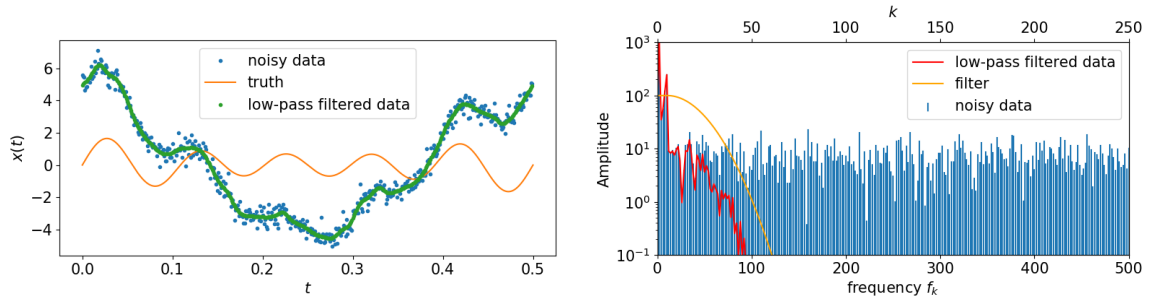


Figure 1.16: Fourier low pass filtering

but it can be combined with a low-pass filter to both smooth and background subtract the data. This is called band-pass filtering. The band can be fairly broad as in the example in Figures 1.16 and 1.17. Or, if you know the expected frequency of the signal, you can choose the band to be tightly focused around the expected frequency.

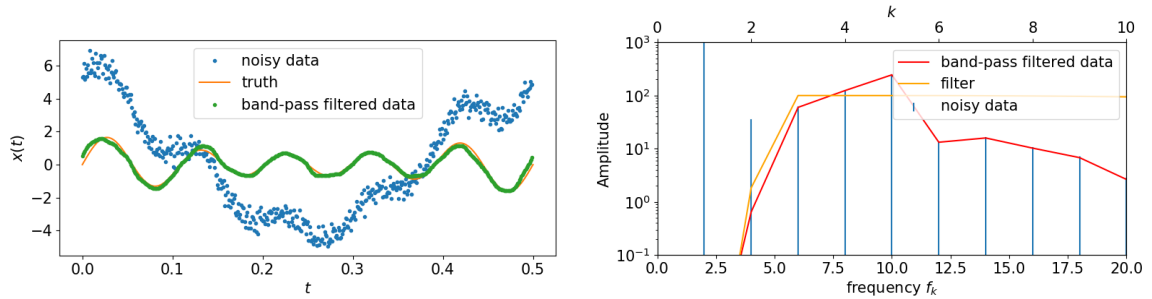


Figure 1.17: Fourier low and high pass filtering

1.11 Auto-correlation

The **autocorrelation** in a time series (or any other series of data events) is useful measure of how well correlated different epochs of the data series are with one another. One can use a **lag k scatter plot**, such as that shown in Figure 1.18, to reveal whether there is autocorrelation present in the data series. One plots all values of $x(t_j)$ against $x(t_{j+k})$, where k is varied to explore the data. A random scatter plot suggests no autocorrelation, while a linear trend suggests that there may be correlation present in the data.

Mathematically, it is similar to the (bivariate) **linear correlation coefficient** that we looked at in Lecture 2. The autocorrelation function is defined as,

$$ACF(k) = \frac{\sum_{j=1}^{n-k} (x_j - \bar{x})(x_{j+k} - \bar{x})}{\sum_{j=1}^n (x_j - \bar{x})^2} = \frac{\sum_{j=1}^{n-k} (x_j - \bar{x})(x_{j+k} - \bar{x})}{N \text{var}(x)} \quad (1.27)$$

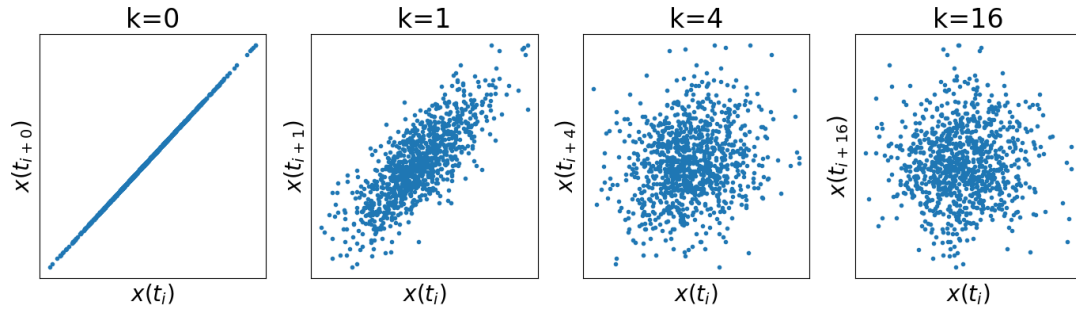


Figure 1.18:

where $\hat{x}$ is the mean of the time series. The parameter k is an integer that defines the **lag time**, and the indices in the above expression refer to the discrete time bins at which the data were obtained.

In terms of mathematical functions the autocorrelation function is defined as

$$ACF(t) = \int x(t')x(t' + t)dt' \quad (1.28)$$

You can visualize the operation as offsetting the data by t and doing the integral (or sum) of the product of the data and offset data at each offset.

A plot of ACF as a function of k is referred to as a **correlogram**, and is used to analyse the correlations that may exist in the time series. For time series with rapidly varying, correlated features, the correlogram will have strong ACF coefficients at small values of k , while series with long-term memory will show strong values at large values of k . **Random noise** – also referred to as **white noise** will produce values of ACF that are close to zero for all k . Periodic variations in the data series will produce periodic variations in the correlogram – remember that k here is **not** a wavenumber. Such periodic variations are best treated with Fourier analysis (see below). Note that $ACF(0) = 1$.

1.12 Correlated noise

As mentioned earlier, random Gaussian noise is expected to have a flat, or 'white' power spectrum.

The ACF is commonly used after regression to check that the fit was a good one. If the model is a good representation of the data, the residuals should be simply random noise, where the residual at one point is independent of the other points. If there is structure in the residuals, e.g. neighbouring points have similar residuals, then there may be some discrepancy between the model and data. The procedure is relatively straightforward:

- one first performs a regression fit of the model to the data, either via χ^2 minimisation or maximum likelihood

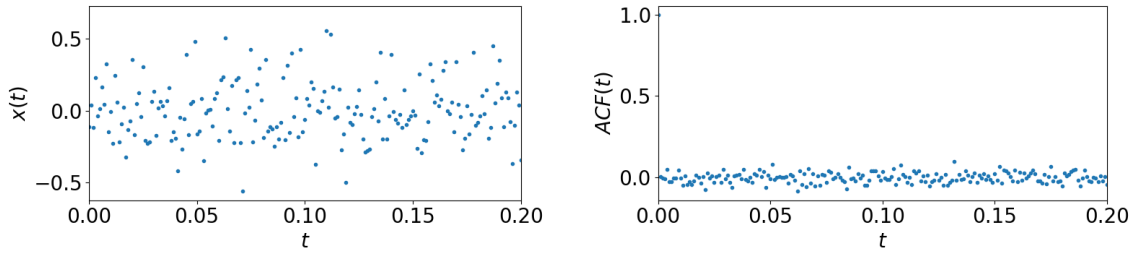


Figure 1.19:

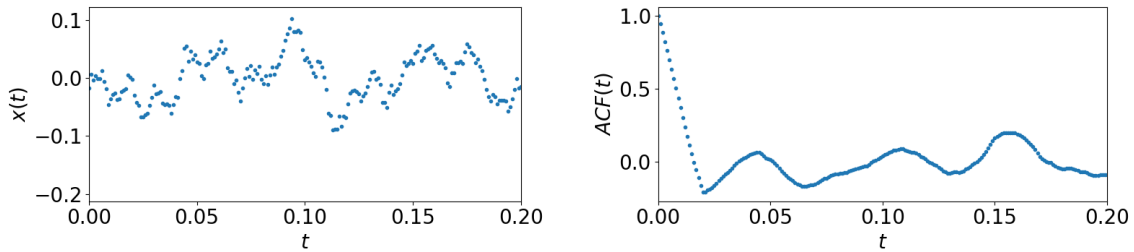


Figure 1.20:

- the fit is then **subtracted** from the data, leaving a series of points that represent the residual scatter around the ‘best-fit’ solution that was found for the model.
- One then performs the ACF on the series of residuals,

$$R = \{(t_1, x_1 - x_{fit}(t_1)), \dots, (t_N, x_N - x_{fit}(t_N))\} \quad (1.29)$$

and inspects the ACF for any significant correlations in the data

This type of analysis is useful for determining if there features of the data that the fitted model somehow missed. For example, we may have fitted a long-period sine function to a time series, by missed the fact that there was a shorter-period sine function superimposed on the long period signal.

For the case of completely uncorrelated data, it can be analytically shown that the distribution for ACF is asymptotically normal, with mean $-1/N$ and variance $1/N$. This is a useful result! We can create a plot of the correlogram, with confidence intervals based on the null hypothesis that the time series has zero correlation. This can be used to test assertions that there are correlations in the time-series. For example, if we are to adopt the classic 95% as our confidence limit, then we would plot error bars of $\pm 2\sigma = 2/\sqrt{N}$. Note however that even with completely uncorrelated data, there is still a 5% chance that and one lag-set will yield a value for the ACF that is above the 2σ expectation, i.e. a 1/20 chance. This means that for every 20 lags, we expect to see a ‘false positive’, so some care must be taken when using such a null hypothesis.

Another problem with analysing the ACF is that the successive values of $ACF(k)$ tend to be similar to one another – i.e. features are not generally only 1 time-bin wide, and this is true for even random features. To account for this, the **large-lag standard error** is adopted, which is given by,

$$\text{var}_{ACF(k)} = \frac{1}{N} \left(1 + 2 \sum_{i=1}^k ACF(i)^2 \right) \quad (1.30)$$

Note now that the variance at any given lag now depends upon the values of ACF that came before it. Typically, the large-lag estimate of the 95% confidence limit is plotted in addition to the asymptotic value based around the variance of $1/N$ when analysing the correlogram.

A consequence of autocorrelation, is that while the sample mean is a valid estimate of the population mean for stationary processes, the variance is not, if autocorrelation is present. In this case we have,

$$\text{var}(\hat{X}) = \frac{\sigma^2}{N} \left[1 + 2 \sum_{k=1}^{N-1} (1 - k/N) ACF(k) \right] \quad (1.31)$$

Although complicated to derive, the qualitative reasoning for this relation is relatively simple: although you have more data points as N increases, the correlation means that these are not truly independent, and so the usual standard deviation in the mean $\sigma/\sqrt{N}$ no longer holds.

So autocorrelation is useful for helping us to uncover repeating trends in a single time series. But what if you have two data sets, and want to see if there are correlations between them? In this case we can simply extend the above process to work on two different data sets, rather than the same data set.

1.13 Cross-correlation

We can also define the **cross-correlation function** between two even spaced data sets $\{x_j\}$ and $\{y_j\}$ as,

$$CCF(k) = \frac{1}{N} \frac{\sum_{j=1}^{N-k} (x_j - \hat{x})(y_{j+k} - \hat{y})}{\sqrt{\text{var}(x) \text{var}(y)}} \quad (1.32)$$

As for the autocorrelation function this can also be defined for mathematical functions:

$$CCF(t) = \int x(t')y(t+t')dt' \quad (1.33)$$

Here, you can visualize the operation as varying the offset t between the two functions and doing the integral (or sum) of the product of the two functions at each offset.

The CCF will be large for values of the offset k which make the values of $\{x_j\}$ are almost the same as $\{y_{j+k}\}$. If $\{x_j\}$ is uncorrelated with $\{y_{j+k}\}$, then the CCF will be close to zero. This can be useful, for example, when trying to determine a shift two spectra. The most likely offset will be the k value that provides the maximum of the $CCF(k)$.

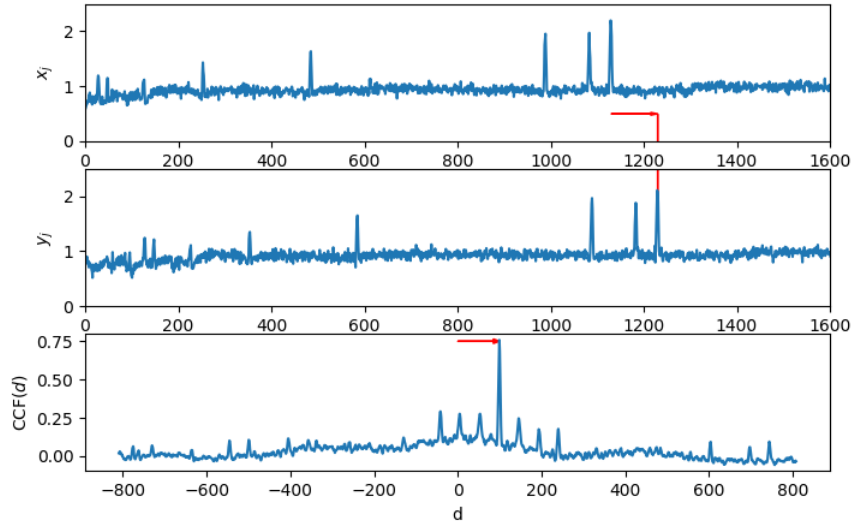


Figure 1.21: The top 2 panels show data x and y . By eye, it is clear that y is very similar to x , but with a delay of about 100 samples. The bottom panel shows the corresponding $CCF(d)$ with a large peak near the offset of 100 samples.

1.14 Convolution

Convolution is a specific way to combine two functions. Given two functions $x(t)$ and $y(t)$ the convolution of x and y is defined as

$$h(t) = x * y = \int x(t')y(t - t')dt' \quad (1.34)$$

For time series data the integral can be replaced by a sum over the measurements

$$h_k = \sum_j x_j y_{k-j} \quad (1.35)$$

This is almost the same as the definition of the cross-correlation function (Equation 1.32, , and $k \rightarrow -k$.

This is clearly very similar to the definition of cross-correlation, and again you can visualize the operation as varying the offset between the two functions and doing the integral (or sum) of the product of the two functions at each offset. However, it is not mean adjusted or normalized by the variance. Also, the offset has the opposite sign, so $t \rightarrow -t$, and the y function must be flipped, $y(t) \rightarrow y(-t)$.

The reasons for these changes, is that with this definition, the Fourier transform of the convolved function is simply the product of the Fourier transform of the two starting functions. So, if we write $X(f) = \text{FFT}(x(t))$, $Y(f) = \text{FFT}(y(t))$ and $H(f) = \text{FFT}(h(t))$, the

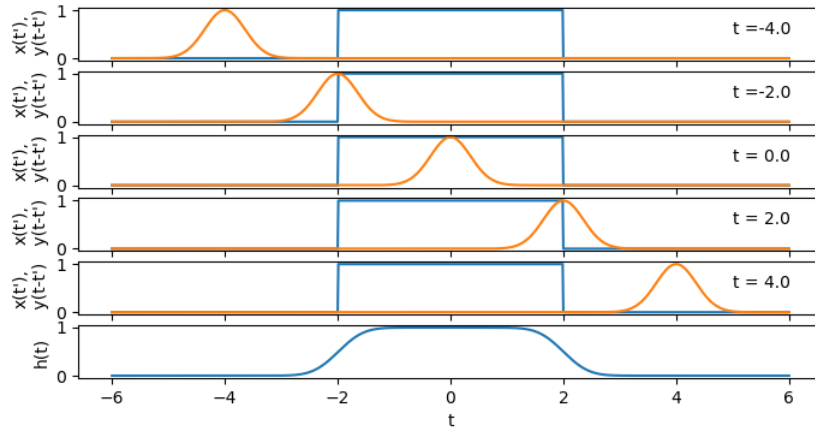


Figure 1.22: The top 5 panels show functions $x(t')$ and $y(t - t')$, with $t = -4, -2, 0, 2, 4$ respectively. The last panel shows the corresponding $h(t)$

convolution is simply

$$H(f) = X(f)Y(f) \quad (1.36)$$

This means that convolution can be carried out very easily using FFTs

$$h(t) = \text{FFT}^{-1}(\text{FFT}(x)\text{FFT}(y)) \quad (1.37)$$

where FFT^{-1} means the inverse Fourier transform. It is often much quicker to calculate a convolution using FFTs rather than simply calculating the integral or sums in the definition of convolution.

Notice that if you set $\{y_k\} = \{x_k\}$ the sum becomes a scaled version of the autocorrelation function, so the ACF is the convolution of the data by itself

$$\text{ACF}(-k) = \frac{\sum x_j x_{-(j+k)}}{N\text{var}(x)} \quad (1.38)$$

This means that the Fourier transform of the ACF is given by

$$\text{FFT}(\text{ACF}) = \frac{A_k A_{-k}}{N\text{var}(x)} = \frac{|A_k|^2}{N\text{var}(x)} \quad (1.39)$$

where $A_k = \text{FFT}(x_j)$. This means that power spectrum is a scaled version of the FFT of the autocorrelation function.

Convolution is important in signal processing in two other important situations:

1. Instrumental effects often mean that a signal whose true shape is a delta function is seen as a point-spread function (PSF). A point source observed through a telescope is seen as an extended image. The true light distribution on the sky is convolved by the PSF

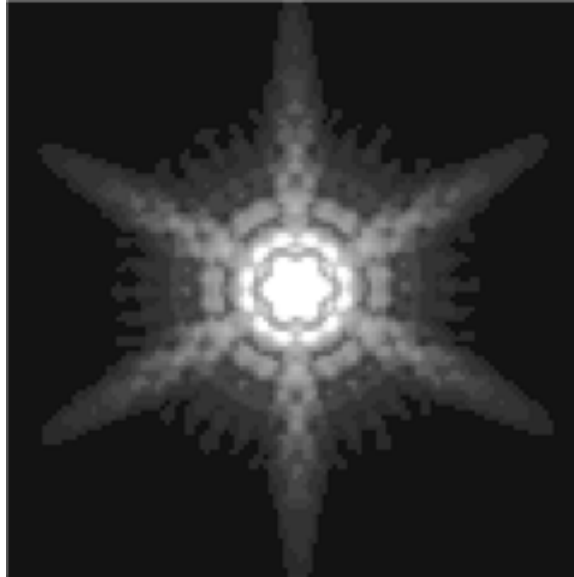


Figure 1.23: An example point-spread function (PSF) for 2 dimensional measurements. A set of point source observed through a system with this point spread function will be seen as a set individual PSFs.

2. The optimal way to detect a signal in noisy data, is to convolve the data by the expected shape of the signal. This process is called **matched filtering**. If you want to find point sources in an image with simple Gaussian noise, it can be shown that using a matched filter will provide the highest signal-to-noise measurements.